

Memo: what this CDP can and can't tell sales

Every figure below is read out of the built warehouse, not estimated.

1. Surface areas

For UI/SDK/CLI/MCP, is attribution deterministic, heuristic, or impossible from this data and why? Point to columns.			
SURFACE	VERDICT	COLUMNS IT RESTS ON	WHY THAT VERDICT
UI	Deterministic	raw_segment_pages.user_id, .path	A logged-in page view carries the user id on the row. 8 of 10 page views attribute; the other 2 are anonymous (user_id null) and are excluded, not guessed.
SDK	Deterministic for 18 of 31 objects, heuristic for 1 more, and impossible to separate from the CLI	raw_experiments.repo_info; raw_experiments.metadata.ci_run_id / .ci / .source; raw_datasets.metadata.source; raw_trace_volume.ingested_gb	repo_info proves the object was logged from code inside a git repo. metadata.source = lambda-prod only suggests it — customer free text, scored 0.60. The CLI wraps the SDK and no column separates them. raw_trace_volume proves the org sends traces programmatically but carries no user_id, so it is org-level only.
CLI	Impossible for usage; heuristic for intent	raw_cli_wizard.user_id; raw_api_keys.name containing "cli"	The wizard is a browser event — opening it installs nothing. A key name is free text, and no object records which key created it. 3 CLI signals exist in total, all intent.
MCP	Impossible for usage; heuristic for intent	raw_docs_mcp_events.action	Copying an install command from the docs is not a connection or a call. 2 signals, both intent.

The remaining **12 of 31 objects** carry neither repo_info nor a source tag and are recorded surface = unknown, never rounded up to UI. raw_prompts has no metadata column at all, so its origin is unknowable by construction.

One absence causes all four verdicts: **no column in any of the 18 files records a client, a user agent, a surface, or an api_key_id**.

2. Resolution quality

How would you measure whether your external→account matching is good? What's your stance on precision vs. recall for a CDP feeding a sales team, and how did that shape your precedence rules?

How I'd measure it. Five metrics, all recomputed by rpt_resolution_quality on every run so drift is visible: **precision** on a monthly steward-labelled sample split by match_method, with a 98% bar for anything a rep sees; **coverage**, today 8 of 11 companies = 73%, though only 7 deliver signals because one is held; **corroboration**, 4 of 8 matches resting on two or more independent keys = 50%; **conflict rate** 1 of 11 = 9% plus ambiguity rate, watched as alarms rather than targets; and **how often reps click "wrong account"**, the only one that measures the thing itself. name_agrees_with_match (6 of 8 = 75%) is a free audit proxy until real labels exist.

Stance: precision, decisively. A bad merge sends a rep another company's news. One of those is enough for a team to stop opening the tool, and that credibility does not come back. A miss sits in a queue where a human can still find it. Recall is deferrable; trust is not.

How that shaped the precedence rules. Four rules, each traceable to a case in this data. (1) Only a key pointing at exactly one account may decide — hooli.com belongs to ACC006 and ACC007, so it decides nothing. (2) LinkedIn slug beats website domain beats LinkedIn-reported domain, because subsidiaries get their own page while sharing a parent's domain. (3) Every other key that matched must agree, or it is a conflict and no account is assigned — sc11 carries Acme's domain with Pied Piper's LinkedIn and gets nothing. (4) A name never decides, and a name pointing elsewhere costs 0.20 and holds the match — sc05 "Hooli Inc" resolves to Hooli XYZ on the slug, so its 3 signals wait for a steward.

3. "% of the account using the CLI"

Can you answer it truthfully today? What would you report and what would you refuse to claim?

What I would report, for every account that has product members. Denominator is known product members.

ACCOUNT	KNOWN MEMBERS	CLI SETUP INTENT	CODE-BASED USAGE (SDK OR CLI)	CLI USAGE SPECIFICALLY
ACC001 Acme Corp	3	0% (0 of 3)	33% (1 of 3)	no number exists
ACC002 Globex	3	33% (1 of 3)	33% (1 of 3)	no number exists
ACC003 Initech	1	0% (0 of 1)	0% at member level; org-level traces prove code use	no number exists
ACC004 Umbrella Labs	1	0% (0 of 1)	100% (1 of 1)	no number exists

Caveats on those numbers. The denominator is *product members*, not company headcount — nothing in the data gives headcount. At Globex a second member showed CLI intent but is not counted: he belongs to two accounts and his browser events carry no org. Initech's code use is proven only at org level (trace ingestion), so evidence_level = org and no individual member is claimed as a user. In fct_account_surface_adoption the CLI usage column is null, not 0 — absent evidence, not evidence of absence.

Can I answer it truthfully? For intent, yes: the percentages above. For usage, no number exists to give — no column records which client made a call, so a CLI usage percentage cannot be computed from these 18 files at any confidence.

What I would refuse to claim. A CLI usage percentage. CLI inferred from `repo_info` — that is code, which is SDK-or-CLI. CLI inferred from a key named `dave-cli` — free text, and keys are not linked to objects. And "% of the account" read as headcount.

`cli_and_mcp_never_claimed_as_usage` enforces the first three in code, and `account_360.adoption_in_plain_english` ships these caveats to the rep next to the number.

4. Two changes

One instrumentation change (product side) and one data addition (CRM/enrichment side) that would most improve this — where captured, where stored.

- **Product instrumentation.** *What:* stamp `client_surface (ui | sdk | cli | mcp)`, client version and `api_key_id` on every write. *Where captured:* at the API gateway, from the client user-agent, the SDK's own header, or the MCP server id — one middleware, no SDK release required. *Where stored:* denormalised onto `experiments`, `datasets` and `prompts` as three columns, plus a new `api_request_log` at request grain for surfaces that create no object. *Effect:* CLI and MCP move from impossible to deterministic, SDK separates from CLI, the 12 unknown-origin objects resolve, and service keys identify themselves instead of hiding behind whoever owns them.
- **CRM data addition.** *What:* required `website_domain` and `linkedin_company_url` on Account, plus a populated `ParentId`. *Where captured:* an enrichment vendor called at account creation and on a refresh schedule, with the field required at save. *Where stored:* on the Salesforce Account object, landing in `raw_salesforce_accounts`. *Effect:* missing identifiers cause most non-matches today — ACC008 has neither, ACC002 no domain, ACC003 and ACC005 no LinkedIn — and the hierarchy lets Hooli and Hooli XYZ roll up without anything being merged.

5. Sales motion

What sales motion does this 360 serve, and how would you know it's useful rather than noise?

The motion is expansion into the existing book, with a product-led feeder alongside. Three plays it supports today: a usage surge on a non-Enterprise plan (Globex +41%, Umbrella +327%, both `pro`) as an upgrade trigger; CLI and MCP intent as an enablement opening, where a rep offers help with a surface someone is already trying to set up; and funding, launch or hiring news as timing.

`net_new_prospects` feeds the second motion — companies we can see but do not sell to, and product sign-ups with no CRM account.

Useful rather than noise, in four measures. Two leading: do reps open the 360, and what share of pushed signals get acted on rather than dismissed. Two lagging: signal-sourced pipeline and expansion ARR, both measured against a **random holdout** of accounts that receive no signals, so the platform cannot take credit for deals that would have closed anyway. The kill rule is agreed in advance: a signal type showing no lift over holdout, or dismissed more than roughly 70% of the time, is cut rather than defended.

6. Which signals drive value

We want to push actionable insights to the sales org and have them work the best accounts with the most important signals. How would you approach identifying what signals actually drive value for the business?

1. **Fix the outcome first**, before looking at any signal: a new opportunity, an upgrade, or an expansion booking within 30–90 days of the signal date. Agreeing that with sales leadership up front is what stops the analysis becoming a search for a flattering metric.
2. **Start collecting history now.** Everything today is a current-state view, so nothing can yet answer "what did a rep see, and when". Snapshot `fct_account_signals` daily — it is already one row per signal in one shape, so it doubles as the feature store and joins to opportunities on `account_id` and date.
3. **Measure per signal type:** lift over a matched baseline controlled for plan and account size, lead time from signal to outcome, and precision at top-N, since reps work a ranked list rather than a population. Report per `signal_type`, because "signals work" is not actionable — "funding news works, tech detections do not" is.
4. **Then replace the v0 rules with learned weights**, keeping the reason visible beside the score. A rep will not act on a ranking they cannot interrogate, and an opaque model that is right 70% of the time loses to a transparent one that is right 60%.
5. **Guard the conclusion:** holdouts so lift is causal rather than correlational, rep dispositions ("wrong account", "not useful") as labels feeding straight back into precision, and a standing commitment to retire signal types that show nothing.